

T12.2 - Organización del Master Dataset

Organización del Master Dataset

Se sabe que la *batch layer* es capaz de *leer el master dataset entero para poder generar las batch views*.

Más allá de eso, sigue siendo necesario organizar los archivos por dos razones:

1. Muchos procesos generan batch views a partir de solo una parte de los datos, no necesariamente del master dataset completo. *Organizar los archivos permite optimizar*.
2. **Small-files problem**. Los sistemas distribuidos sufren cuando los datos están dispersos en muchos archivos chicos.

Particionado por verticales

Es un mecanismo de organización que consiste en *dividir los datos a partir de sus características*.

En una plataforma de streaming (tipo Spotify/Netflix), el master dataset recibe muchos tipos de eventos distintos.

El particionado vertical consiste en separar cada tipo de evento en su propia carpeta. Cada carpeta es un vertical, agrupa un mismo tipo de hecho o evento:

```
master-dataset/  
├── content-interactions/  
├── logins/  
├── likes/  
└── search-actions/
```

→ El almacenamiento batch debería permitir particionar la data para procesar solamente aquella que sea útil para el proceso.

El almacenamiento batch era dónde y cómo se guarda el Master Dataset. Es la infraestructura donde vive el Master Dataset.

→ El particionado por verticales *permite a hacer los trabajos de la batch layer más eficientes*, como así también *optimiza los recursos del filesystem y recursos de cómputo*.

→ Dentro de un *mismo vertical*, se puede *particionar por tiempo*.

→ *Reduce* transferencias innecesarias.

Si todo estuviera mezclado en un único archivo grande (con logins + likes + ...) y solo se quieren analizar los logins, igualmente habría que leer/transferir el archivo entero con bastantes datos que no sirven para la ocasión. Tener un archivo dedicado para un evento en específico a analizar es lo ideal porque no hay desperdicio de datos.

Áreas y consolidación

Un *problema común* en los sistemas de cómputo y almacenamiento distribuido es el **small-files problem**.

Los datos llegan a un sistema con arquitectura lambda *de a goteo*, en *muchos archivos chicos*.

Esto es un problema en sistemas distribuidos como HDFS porque:

- HDFS *divide el cómputo en tareas según los bloques*.

HDFS guardaba los archivos en bloques. Guardar un archivo implica que se corte en pedazos de tamaño fijo (bloques) y estos se reparten entre los distintos nodos del cluster. Recordar que puede haber duplicados.

- Los *bloques tienen todos el mismo tamaño*, el último puede ser igual o más chico.
- Cada archivo implica *tantas tareas como bloques tenga*.

Esto se basa en el paralelismo.

Los bloques ya están repartidos físicamente entre los nodos del cluster. Cada bloque genera una tarea, y las tareas se reparten entre los nodos. En vez de mover los datos hacia donde está la tarea (caro, satura la red), el sistema hace lo inverso: manda el cómputo hacia donde están los datos. Lanza una tarea por bloque y la ejecuta, idealmente, en el nodo que ya tiene ese bloque guardado localmente.

- *Procesar un mismo volumen* de datos *distribuido en muchos archivos chicos* necesita *muchas más tareas* que un solo archivo grande → cada tarea implica un *overhead*, un costo extra.

Para *evitar la dispersión de archivos* y *organizar las particiones verticales*, se pueden *imponer áreas* dentro del Master Dataset.

- **Landing:** acá *llegan los datos en archivos chicos*. Pueden haber archivos con un único registro cada uno, para evitar esto se pueden acumular en colas.
- **Main:** *acá los archivos están organizados en verticales y consolidados*. Los archivos chicos se terminan consolidando en archivos grandes. Este proceso de consolidación se apoya en la misma infraestructura que va a generar las batch views.